

INTRODUCTION TO PROGRAMMING

UNIT I - Introduction to Programming and Problem Solving

History of Computers, Basic organization of a computer: ALU, input-output units, memory, program counter, Introduction to Programming Languages, Basics of a Computer Program: Algorithms, flowcharts (Using Dia Tool), pseudo code. Introduction to Compilation and Execution, Primitive Data Types, Variables, and Constants, Basic Input and Output, Operations, Type Conversion, and Casting.

Problem solving techniques: Algorithmic approach, characteristics of algorithm, Problem solving strategies: Top-down approach, Bottom-up approach, Time and space complexities of algorithms.

History of Computers:

- To perform computations we use machines.
- The earliest tool is abacus, a device used in china and United States today as a hand held calculator.
- They require human involvement.
- The first attempt was made by Charles Babbage in creating a programmable computing machine in England in 1822.
- Ada Byron the daughter of famous poet Lord Byron developed the instruction set for the machine.
- The machine that Babbage called it was analytical engine.
- In 1937, Dr. John V Atanasoff and a graduate student, Clifford Berry developed a machine called ABC using electrical components at Iowa State University.
- The goal internally was to store a set of instructions. In 1939, Dr. John W Mauchly from Moore college of Engineering at Pennsylvania University along with Presper Eckert developed a computer called ENIAC (Electrical Numerical Integrator And Computer).
- The fund was given by US government and was completed in 1946.
- It consists of 18000 vacuum tubes and can perform 5000 additions and 360 multiplications in one second.
- At the same time Mark developed a computer by name Mark I at Harvard University using mechanical relays and completed in 1944.
- But it could perform only 6 multiplications in one second.

- The final goal of storing the program in computer was achieved in 1949 at Cambridge University with the successful operation of EDSAC (Electronic Delayed Storage Automatic Computer).
- This contains memory where it could store data.
- The principles for storing the data and retrieve data and instructions has been proposed by John Von Neumann.

Definition of Computer and its Characteristics:

Definition of Computer:

Computer is an electronic device and it takes the data and instructions as Input and process that data according to the user instructions and produces the result as output.

Characteristics of Computer: Following are the characteristics of computer.

1. Speed:

A computer can do the calculations very fastly. A computer can do the 3 to 4 millions of simple arithmetic calculations per a second. A Human can do a work with in the days that can be done by the computer with in the milliseconds. The speed of a computer can measured in terms of milliseconds, micro seconds, nanoseconds and picoseconds.

2. Accuracy:

The computer is capable of doing calculations with a high accuracy. Moreover, it does every calculation with the same accuracy.

3. Diligence:

The computer is capable of doing work for hours and that too without creating any error. It does not feel tiredness or lack of concentration.

4. Versatility:

Versatility refers to the capability of a computer to perform different kinds of works with same Accuracy and efficiency.

5. Memory Capability:

The computer does not differentiate the type of data to be remembered. It is a capable of storing and recalling any amount of information in its secondary storage device. The memory can be measured in terms of BITS, Bytes, Kilo Bytes, Mega Bytes, Giga Bytes, etc.

6. Intelligence:

The present day computer does not possess intelligence of its own. Its intelligence quotient is zero. Computer is obedient servant to its master and works according to the user commands.

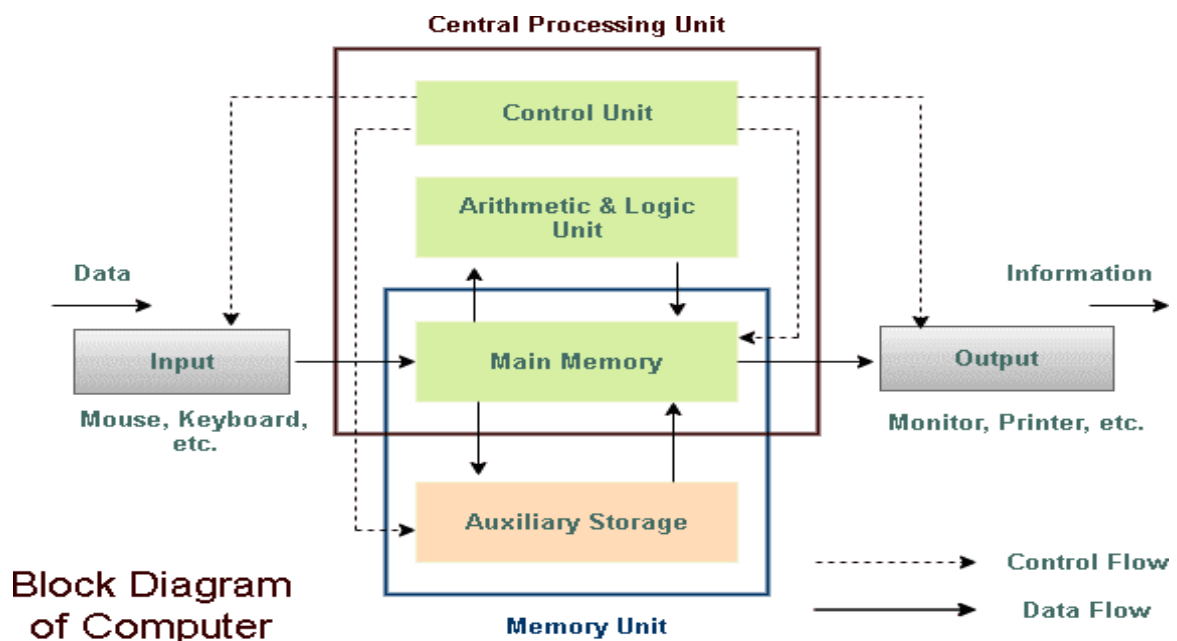
Basic Organization of Computer:

A computer is an electronic device that takes data and instructions as input, processes data and provides output.

BLOCK DIAGRAM OF COMPUTER:

The Block diagram of computer includes 3 Parts.

1. Input Unit.
2. Central Processing Unit.
3. Output Unit.



1. INPUT UNIT

- Computers need to receive data and instruction in order to solve any problem.
- We need to input the data and instructions into the computers.
- The input unit consists of one or more input devices.

- Keyboard is the one of the most commonly used input device.
- Other commonly used input devices are the Mouse, Scanner, Microphone, etc.
- All the input devices perform the following functions.
- Accept the data and instructions from the outside world.
- Convert it to a form that the computer can understand.

Supply the converted data to the computer system for further processing.

2. CENTRAL PROCESSING UNIT (CPU):

- CPU is the brain of any computer system.
- It is just like brain that takes all major decisions, makes all sorts of calculations and directs different parts of the computer functions by activating and controlling the operations.

ARITHMETIC LOGICAL UNIT (ALU)

- After you enter data through the input device it is stored in the primary storage unit.
- The actual processing of the data and instruction are performed in Arithmetic Logical Unit.
- The major operations performed by the ALU are addition, subtraction, multiplication, division, logic and comparison.
- Data is transferred to ALU from storage unit when required.
- After processing the output is returned back to storage unit for further processing or getting stored.

CONTROL UNIT (CU)

- The next component of computer is the Control Unit, which acts like the supervisor seeing that things are done in proper fashion.
- Control Unit is responsible for coordinating various operations using time signal.
- The control unit determines the sequence in which computer programs and instructions are executed.
- Things like processing of programs stored in the main memory, interpretation of the instructions and issuing of signals for other units of the computer to execute them.

- It also acts as a switch board operator when several users access the computer simultaneously.
- Thereby it coordinates the activities of computer's peripheral equipment as they perform the input and output.

MEMORY OR STORAGE UNIT

- The storage unit of the computer holds data and instructions that are entered through the input unit, before they are processed.
- It preserves the intermediate and final results before these are sent to the output devices.
- It also saves the data for the later use.
- The various storage devices of a computer system are divided into two categories.

a) Primary Storage

- Stores and provides very fast.
- This memory is generally used to hold the program being currently executed in the computer, the data being received from the input unit, the intermediate and final results of the program.
- The primary memory is temporary in nature.
- The data is lost, when the computer is switched off.
- In order to store the data permanently, the data has to be transferred to the secondary memory.
- The cost of the primary storage is more compared to the secondary storage.
- Therefore, most computers have limited primary storage capacity.

b) Secondary Storage

- Secondary storage is used like an archive. It stores several programs, documents, data bases etc.
- The programs that you run on the computer are first transferred to the primary memory before it is actually run.
- Whenever the results are saved, again they get stored in the secondary memory.
- The secondary memory is slower and cheaper than the primary memory.
- Some of the commonly used secondary memory devices are Hard disk, CD, etc.,

3. OUTPUT UNIT

- The output unit of a computer provides the information and results of a computation to outside world.
- Printers, Visual Display Unit (VDU) are the commonly used output devices.
- Other commonly used output devices are Speaker, Headphone, Projector, etc.

PROGRAM COUNTER

- A register is a small place for data holding that the processor uses.
- Program counter is just a small register that could hold 8 bit or 16 bit data.
- It depends on the processor that process data either 8 bit or 16 bit.
- In computing, a program is a specific set of ordered operations for a computer to perform.
- An instruction is an order given to a computer processor by a program.
- Within a computer, an address is a specific location in memory or storage.
- A program counter is a register in a computer processor that contains the address (location) of the next instruction being executed by the processor.
- As each instruction gets fetched, the program counter increases its stored value by 1.
- After each instruction is fetched, the program counter points to the next instruction in the sequence.
- When the computer restarts or reset, the program counter normally reverts to 0.
- It is used to keep track of all instructions that are to be processed.
-

INTRODUCTION TO PROGRAMMING LANGUAGES:

- The computer operations are controlled by a set of instructions called **computer program**. These instructions are written to tell the computer about

- What operation to perform
- Where to locate data
- How to present the results
- When to make certain decisions
- Programming is the process of writing the instructions that the computer can respond to and that the other programmers can understand.
- The set of instructions that are used to construct a program are **Programming languages**.
- The communication between two parties needs language. The language used to communicate the computer instructions is known as **programming language**.
- There are different levels of programming languages. They are
 - Machine language (low level languages)
 - Assembly level language(symbolic languages)
 - Procedure oriented language(high level languages)
 - Object Oriented Languages

MACHINE LANGUAGE

- ✓ Machine language is referred as **first generation** languages.
- ✓ It is also known as low level language.
- ✓ It was introduced in the year 1940.
- ✓ Computer is an electronic device that can understand only binary codes hence the computer instructions and data are written using binary codes 1 and 0.
- ✓ The binary code is called **machine code or machine language**.
- ✓ All computer designs are not same because they have their own machine language and they cannot understand any language like English, Telugu, Tamil etc except machine language.
- ✓ There are some problems with the machine language.
 - ✓ It is difficult to understand and remember the various combinations of 0's and 1's for instructions and data.
 - ✓ Writing error free instructions is a slow process.
 - ✓ The user cannot communicate with other machine because every machine has its own machine language

ASSEMBLY LANGUAGE (SYMBOLIC LANGUAGES)

- ✓ The assembly language is referred as **second generation** languages.
- ✓ It was introduced in 1950.
- ✓ In assembly language, the machine language 0's and 1's are replaced by mnemonic codes.
- ✓ The assembly language consists of series of instructions and mnemonics.
- ✓ These mnemonics corresponds to executable instructions which are followed by zero or more operands.
- ✓ The mnemonic code is called **operation code or opcode** which specifies the operation to be performed based on given operands.
- ✓ During the execution of the program, an **assembler** is used.
- ✓ The assembler is a software program that takes assembly level language program as input and produces machine code as output.
- ✓ The programmers even today they prefer assembly language for performing various tasks.
 - ✓ To initialize and test the system hardware
 - ✓ To write patches for disassembling viruses
 - ✓ To attain optimization
 - ✓ For direct interaction with hardware
 - ✓ To maximize limited resource utilization
 - ✓ For high security

Advantage of assembly language over machine language

- ✓ easy to debug because we can locate and identify syntax
- ✓ easy to develop computer application
- ✓ operates very efficiently

HIGH LEVEL LANGUAGES

- ✓ These languages easily modify the program to reduce the number of operations.
- ✓ They are easy to use, portable and abstract when compared to low level languages.
- ✓ They use logic to solve the problem.
- ✓ The high level languages are executed by using a compiler or an interpreter.
- ✓ The high level languages are classified into three categories.

- ✓ Procedure oriented languages (third generation)
- ✓ Problem oriented languages (fourth generation)
- ✓ Natural languages (fifth generation)

Procedure oriented languages:

- ✓ The high level languages designed to solve general purpose problems are called **procedural languages or third generation** languages.
- ✓ They are designed to express logic and procedure of a problem.
- ✓ The advantage of third generation languages is they are **portable**.
- ✓ We can use any compiler or interpreter and create object code.
- ✓ The third generation languages are considered as domain **specific** because they are designed to develop specific applications.

Problem oriented languages

- ✓ These languages are used to solve specific problems.
- ✓ They are also known as **fourth generation** languages.
- ✓ These languages have reduced programming efforts and over all cost of software development.
- ✓ These languages use **visual or text** environments.
- ✓ A single statement in fourth generation is equivalent to multiple statements in third generation for a task.
- ✓ The programmer uses **drag and drop toolbar** for creating various items.
- ✓ These languages are used to facilitate **faster environment** for application development.

Natural languages

- ✓ These languages are known as **fifth generation** languages.
- ✓ These languages are designed to make computer to behave like **expert** and solve problems.
- ✓ The programmer specifies both the problem and its constraints for problem solving.
- ✓ These languages are used to develop **artificial intelligence and expert systems**.
- ✓ The fifth generation languages allow users to communicate with the

computer in simple and easy manner.

OBJECT ORIENTED PROGRAMMING

- ✓ It focuses on developing software based on their component objects.
- ✓ The components interact with each other to provide functionality of the software.
- ✓ In OOP, the software is broken down into components consists of data attributes and methods that operate on data.
- ✓ The components are complete by themselves and are re-usable. The terms that commonly associated with OOPS are
- ✓ Class is the basic building block of the OOP, a class consists data attributes and methods that operate on data.
- ✓ Object is runtime instance of the class. An object has a state, a defined behavior and unique identity.
- ✓ The state of the object is represented by the data defined in the class.
- ✓ Class is template for object.
- ✓ Abstraction, Encapsulation, Inheritance and Polymorphism are unique features of OOP.

BASICS OF A COMPUTER PROGRAM Algorithm:

- ✓ An algorithm is step by step procedure to solve a problem.
- ✓ An algorithm is defined as a finite sequence of instructions each of which has a clear meaning and can be performed with a finite amount of effort in a finite amount of time.
- ✓ The algorithm word originated from the **Arabic word “Algorism”** which is linked to the name of the Arabic mathematician **AI Khwarizmi**.
- ✓ He is considered to be the first algorithm designer for adding numbers.

Properties of Algorithms

1. **Finiteness:** An algorithm must terminate after a finite number of steps.
2. **Definiteness:** The steps of the algorithm must be precisely defined or unambiguously specified.
3. **Generality:** An algorithm must be generic enough to solve all problems of a

particular class.

4. **Effectiveness:** The operations of the algorithm must be effective such that it must easily converted into machine code in a finite amount of time.
5. **Input-Output:** The algorithm must have zero, one or more inputs and one or more outputs.

Algorithm for adding two numbers:

Step 1: Start

Step 2: Read a, b

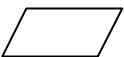
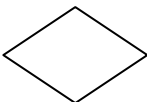
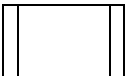
Step 3: $c \leftarrow a+b$

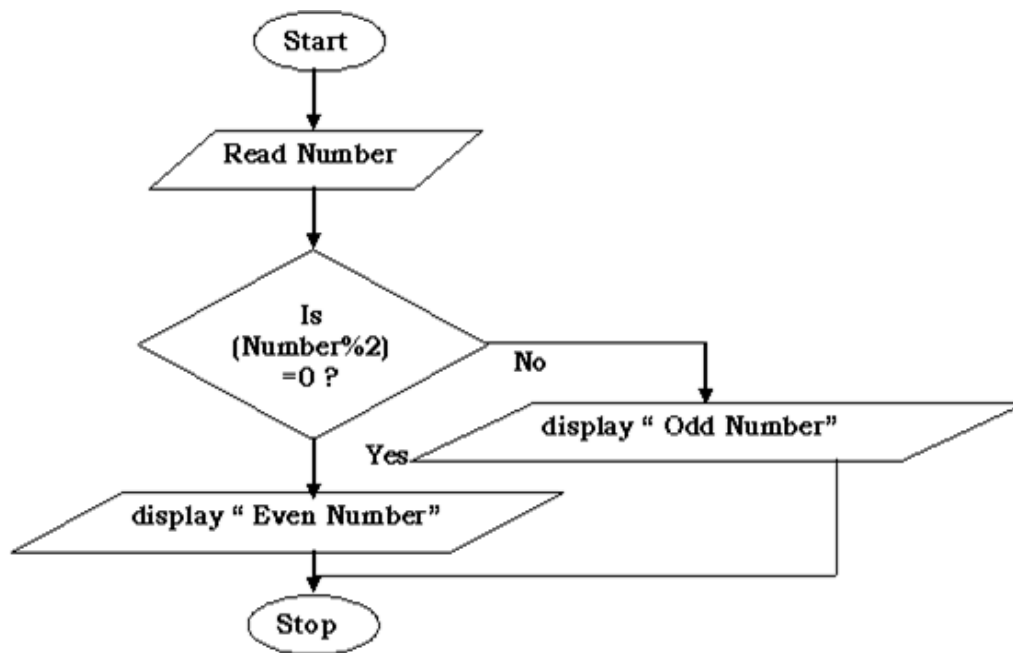
Step 4: Write C

Step 5: Stop.

FLOW CHART:

- ✓ Flow chart is diagrammatic representation of an algorithm.
- ✓ It is built using different types of boxes of symbols.
- ✓ The operation to be performed is written in the box.
- ✓ All symbols are interconnected by arrows to indicate the flow of information and processing.

Oval		Terminal	Start/stop/begin/end symbol
Parallelogram		Input / Output	Making data available for processing (input) or recording of the processed information (output).
Rectangle		Process	Any processing to be performed. An assignment operation normally represented by this symbol.
Diamond		Decision	Decision or switching type of operations that determines which of the alternative paths is to be followed.
Circle		Connector	Used for connecting different parts of flow chart.
Arrow		Flow	Joins two symbols and also represents executions flow.
Bracket with broken line		Annotation	Descriptive comments or explanations
Double sided rectangle		Predefined process	Modules or subroutines given elsewhere.

Flowchart for Even or Odd

PSUEDO CODE

- ✓ The enhanced form of an algorithm that focuses on logic of the program is called pseudo code.
- ✓ It refers to false code written in a programming language.
- ✓ It does not have any syntax for representation but normally used structured English language for representation.
- ✓ It helps programmer to understand the basic logic before writing the final code in a particular language.

EXAMPLE

To add two numbers

1. DEFINE n1, n2, result
2. READ n1, n2
3. COMPUTE result – n1 + n2
4. DISPLAY result
5. EXIT

Advantages of Pseudo Code

1. It is easier to modify the program logic when necessary.
2. It takes less time for writing it
3. It takes less effort for writing it
4. Converting it to programming code is easier.
5. It does not have any syntax therefore we can write it as we like.

Disadvantages of Pseudo Code

1. A graphic representation of program logic is not available.
2. There is no standard rule for representation of it.
3. Several programmers follow different styles to write it.
4. Communication problem occurs due to lack of standards.
5. For beginners it is difficult to follow.

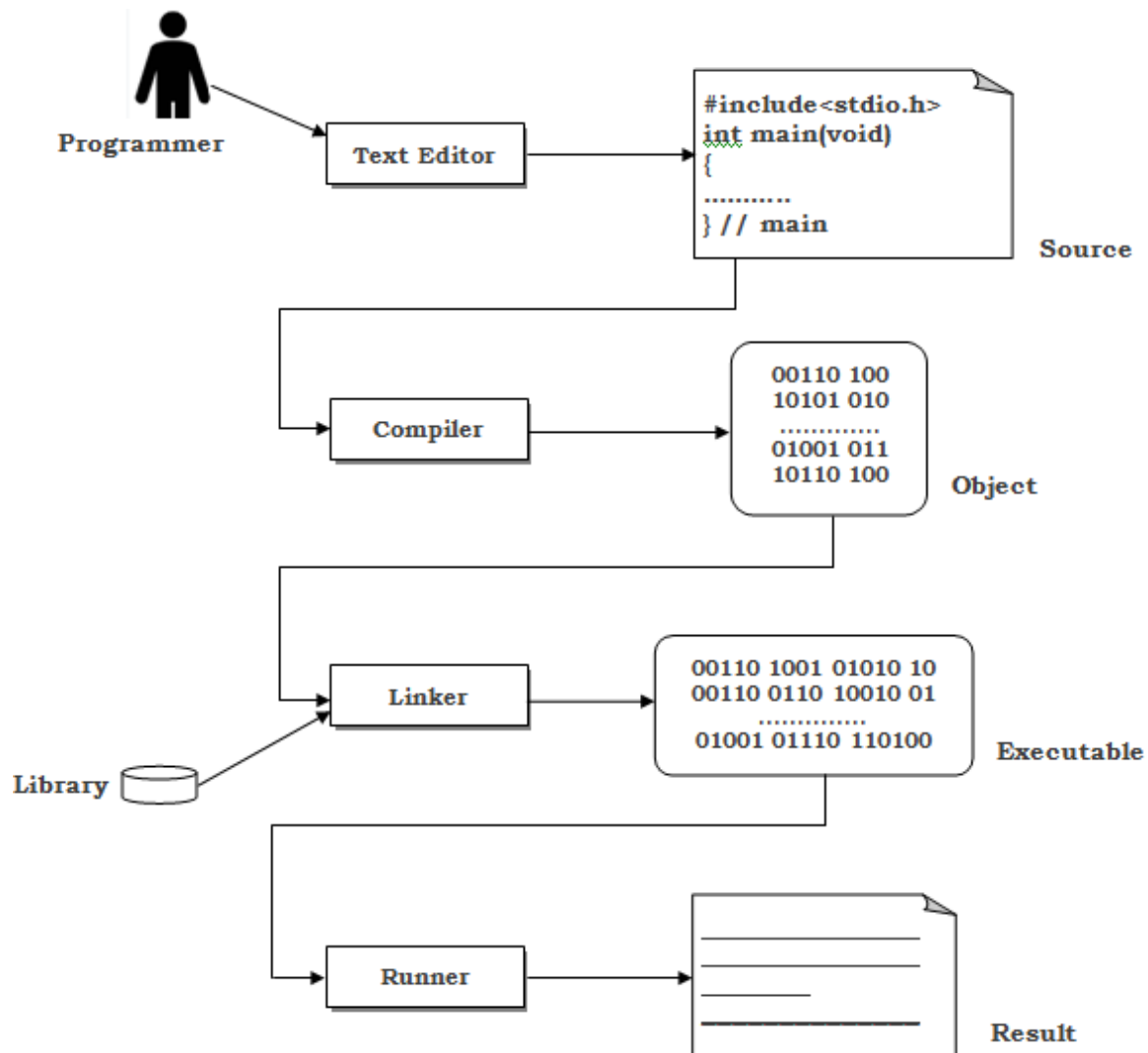
INTRODUCTION TO COMPILATION AND EXECUTION

- ✓ The procedure for converting a program written in C language into machine language is shown below.
- ✓ There are four steps in this process:
 - ✓ Writing and editing program
 - ✓ Compiling the program
 - ✓ Linking the program with required libraries
 - ✓ Executing the program

Writing and Editing Programs

- ✓ Text editor is software used for writing the programs.
- ✓ It helps to enter, change and store character data.
- ✓ The main difference between text processing and program writing is programs are written using lines of code and text processing is done with characters and lines.
- ✓ The text editor is a word processor which is included in the compiler

- ✓ After writing a program we save it on the disk which is input to the compiler and it is known as **source file**.



Compiling Programs

- ✓ The job of the compiler is to convert the source file into machine language.
- ✓ There are two separate programs in C compiler:
 - ✓ Preprocessor
 - ✓ Translator
- ✓ The preprocessor reads the source code and prepares it for the translator.
- ✓ It scans for preprocessor commands and tells to replace the code.
- ✓ The result of preprocessing is translation unit.
- ✓ The translator does the actual work of converting the program to machine language.
- ✓ The translator reads translation unit and produces object module as

output

- ✓ This is not yet ready to run because it needs required functions to include.

Linking Programs

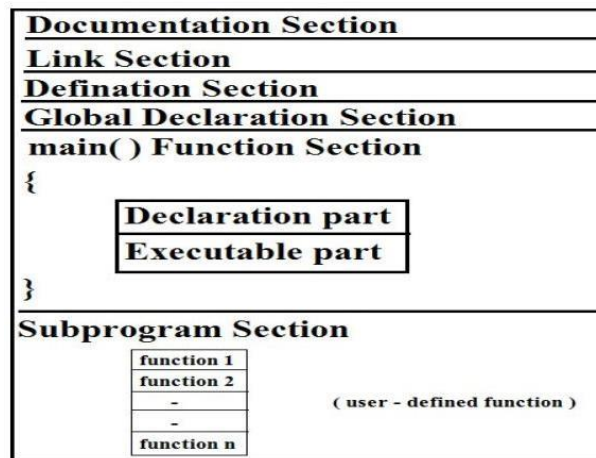
- ✓ C program is made up of many functions, some we write and some they are part of the source program.
- ✓ There are other functions like input/output and mathematical libraries that are attached to source program
- ✓ The linker assembles all these functions to make it finally executable.

Executing Programs

- ✓ The program is ready for execution if it is linked with necessary routines.
- ✓ To execute the program we use OS command run to load the program in main memory and execute it.
- ✓ The loading activity is done by loader.
- ✓ The program execution means
 - ✓ Reads data for processing from file or from user
 - ✓ Process the data
 - ✓ Prepare the output for the user

Structure of a C Program:

Following is the structure of a C Program.



Documentation Section:- The documentation section consists of a set of comment lines giving the name of the program, the author and other details, which the programmer would like to use later.

There are two types of comment lines as follows.

1. Single line comment using the symbol // -----.
Example: // It is c Programming
2. Multiline comment using the symbol /* ----- */
Example: /* it's the c program
 To learn and practice */

The link section:- It provides instructions to the compiler to link functions from the system library. C compiler provide plenty of library like stdio.h,conio.h,math.h all the functions which are defined by this libraries must be attach to the current programs in the following way.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
```

#include provides the link between current program to the library function.

Definition Section:- The definition section defines all symbolic constants.Using the #define symbol.

Example:

```
#define PI 3.141
#define X 20
```

Global declaration section:- There are some variables that are used in more than one function. Such variables are called global variables and are declared in the global declaration section that is outside of all the functions. This section also declares all the user-defined functions.

Main function section:- Every C program must have one main() function section.This section contains two parts, **declaration part** and **executable part**.

The declaration part declares all the variables used in the executable part.

There is at least one statement in the executable part. These two parts must appear between the opening and the closing braces. The program execution begins at the opening brace and ends at the closing brace. The closing brace of the main function section is the logical end of the program. All statements in the

declaration and executable parts end with a semicolon(;).

Subprogram section:- contains all the user-defined functions that are called in the main function. User-defined functions are generally placed immediately after the main function, although they may appear in any order.

All sections, except the main function section may be absent when they are not required.

Ex:

1. Write a C Program to print welcome message.

```
#include<stdio.h>

void main()

{

printf(“WELCOME”);

}
```

OUTPUT: WELCOME

2. Write a C program for addition of 2 numbers.

```
#include<stdio.h>

void main( )

{

int a,b,c;

clrscr( );

a=100;

b=200;

c=a+b;

printf(“\n sum=%d”,c);

}
```

OUTPUT: sum=300

C Character Set

As every language contains a set of characters used to construct words, statements, etc., C language also has a set of characters which include **alphabets**, **digits**, and **special symbols**. C language supports a total of 256 characters. C language character set contains the following set of characters...

1. Alphabets
2. Digits
3. Special Symbols

Alphabets

C language supports all the alphabets from the English language. Lower and upper case letters together support 52 alphabets.

lower case letters - **a to z**

UPPER CASE LETTERS - **A to Z**

Digits

C language supports 10 digits which are used to construct numerical values in C language.

Digits - **0, 1, 2, 3, 4, 5, 6, 7, 8, 9**

Special Symbols

C language supports a rich set of special symbols that include symbols to perform mathematical operations, to check conditions, white spaces, backspaces, and other special symbols.

Special Symbols - **~ @ # \$ % ^ & * () _ - + = { } [] ; : ' " / ? . > , < \ | tab newline space NULL bell backspace verticaltab etc.,**

TOKENS:

A token in C can be defined as the smallest individual element of the C programming language that is meaningful to the compiler. It is the basic component of a C program.

Following are different types of Tokens available in C language.

1. **Keywords**
2. **Identifiers**
3. **Constants**
4. **Strings**
5. **Special Symbols**
6. **Operators**

1. **Keywords:** The [keywords](#) are pre-defined or reserved words in a programming language. Each keyword has a specific meaning in C language and they can't be used for any other purposes which they define. Since keywords are referred names for a compiler. C language supports 32 keywords which are given below:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

2. **Identifiers:** A name in C program is called as Identifier that may be variable name, array name, function name, structure name, etc. Identifiers are the user-defined names consisting of C character set. Identifiers are used to identify a particular element in a program. Each Identifier must have a unique name. following are the Rules for writing Identifiers:

- They must begin with a letter or underscore(_).
- They must consist of only letters, digits, or underscore. No other special character is allowed.
- It should not be a keyword.

- It must not contain white space.
 - It should be up to 31 characters long as only the first 31 characters are significant.
3. **Constants:** The constants refer to the variables with fixed values. They are like normal variables but with the difference that their values can not be modified in the program once they are defined. “const” is a keyword which is used to define the constants.

Example:

```
const int a=10;
```

```
const float x=4.567;
```

4. **Strings:** String is a collection of Characters enclosed with in the double quotes and end with a null character ('\0'). This null character indicates the end of the string.

Example:

```
char str[ ]="welcome";
```

```
char str[10]="hi friends";
```

5. **Special Symbols:** The following special symbols are used in C having some special meaning and thus, cannot be used for some other purpose. Some of these are listed below:
- **Brackets[]:** Opening and closing brackets are used as array element references. These indicate single and multidimensional subscripts.
 - **Parentheses():** These special symbols are used to indicate function calls and function parameters.
 - **Braces{}:** These opening and ending curly braces mark the start and end of a block of code containing more than one executable statement.
 - **Comma (,):** It is used to separate more than one statement like for separating parameters in function calls.
 - **Colon(:):** It is an operator that essentially invokes something called an initialization list.
 - **Semicolon(;):** It is known as a statement terminator. It indicates the end of one logical entity. That's why each individual statement must be ended with a semicolon.
 - **Asterisk (*):** It is used to create a pointer variable and for the multiplication of variables.
 - **Assignment operator(=):** It is used to assign values and for logical operation validation.
 - **Pre-processor (#):** The pre processor is a macro processor that is used automatically by the compiler to transform your program before actual compilation.
 - **Period (.):** Used to access members of a structure or union.

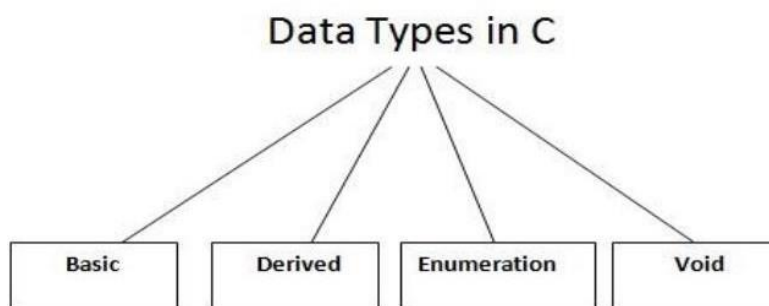
6. **Operators:** Operator is a Symbol which is used to perform either Mathematical or Logical

Operations.

Ex: +, -, *, <, >, etc.

Data Types/Types:

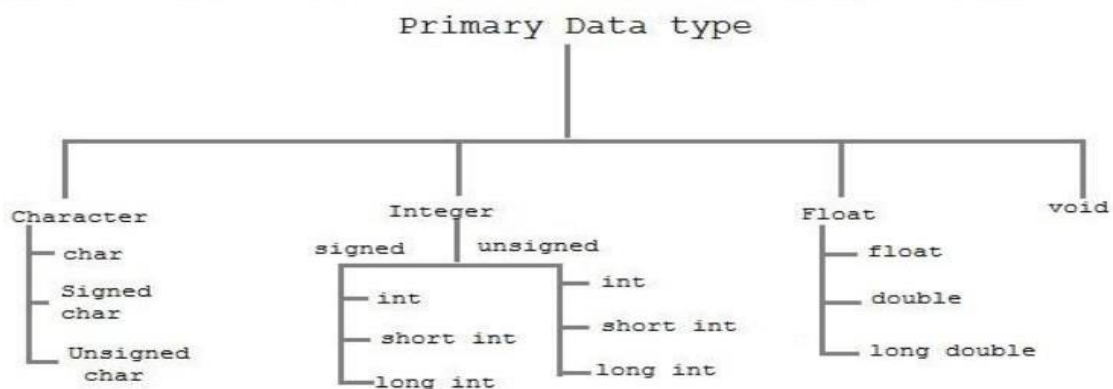
To store data the program must reserve space which is done using datatype. A datatype is a keyword/predefined instruction used for allocating memory for data. A data type specifies the type of data that a variable can store such as integer, floating, character etc. It used for declaring/defining variables or functions of different types before to use in a program.



There are 4 types of data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	enum
Void Data Type	void

Note: We call Basic or Primary data type.



The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals. The memory size of basic data types may change according to 32 or 64 bit operating system. Let's see the basic data types. Its size is given according to 32 bit architecture

Size and Ranges of Data Types with Type Qualifiers

Type	Size (bytes)	Range	Control String
char or signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c

int or signed int	2	-32768 to 32767	%d or %i
unsigned int	2	0 to 65535	%u
short int or signed short int	1	-128 to 127	%d or %i
unsigned short int	1	0 to 255	%d or %i
long int or signed long int	4	-2147483648 to 2147483647	%ld
unsigned long int	4	0 to 4294967295	%lu
float	4	3.4E-38 to 3.4E+38	%f or %g
double	8	1.7E-308 to 1.7E+308	%lf
long double	10	3.4E-4932 to 1.1E+4932	%Lf

The following are some main primitive data types in C:

Integer Data Type

The integer datatype in C is used to store the integer numbers (any number including positive, negative and zero without decimal part). Octal values, hexadecimal values, and decimal values can be stored in int data type in C. We use [int keyword](#) to declare the integer variable:

- **Range:** -32768 to 32767
- **Size:** 2 bytes
- **Format Specifier:** %d

Syntax for declaring the Integer variables:

int var_name;

Ex:

int a;

int x,y;

The integer data type can also be used as

1. **unsigned int:** Unsigned int data type in C is used to store the data values from zero to positive numbers but it can't store negative values like signed int.
2. **short int:** It is lesser in size than the int by 2 bytes so can only store values from -32,768 to 32,767.
3. **long int:** Larger version of the int datatype so can store values greater than int.
4. **unsigned short int:** Similar in relationship with short int as unsigned int with int.

***Note:** The size of an integer data type is compiler-dependent. We can use [sizeof operator](#) to check the actual size of any data type.*

Character Data Type

Character data type allows its variable to store only a single character. The size of the character is 1 byte. It is the most basic data type in C. It stores a single character and requires a single byte of memory in almost all compilers. The **char keyword** is used to declare the variable of character type.

- **Range:** (-128 to 127) or (0 to 255)
- **Size:** 1 byte
- **Format Specifier:** %c

Syntax for declaring the Character variables:

char var_name;

Ex:

char a;


```
char x,y;
```

Float Data Type

In C programming [float data type](#) is used to store floating-point values. Float in C is used to store decimal and exponential values. It is used to store decimal numbers (numbers with floating point values) with single precision. The **float keyword** is used to declare the variable as a floating point:

- **Range:** 1.2E-38 to 3.4E+38
- **Size:** 4 bytes
- **Format Specifier:** %f

Syntax for declaring the floating point variables:

```
float var_name;
```

Ex:

```
float a;
```

```
float x,y;
```

Double Data Type

A [Double data type](#) in C is used to store decimal numbers (numbers with floating point values) with double precision. It is used to define numeric values which hold numbers with decimal values in C. The double data type is basically a precision sort of data type that is capable of holding 64 bits of decimal numbers or floating points. Since double has more precision as compared to that float then it is much more obvious that it occupies twice the memory occupied by the floating-point type. It can easily accommodate about 16 to 17 digits after or before a decimal point. The variable can be declared as double precision floating point using the **double keyword**:

- **Range:** 1.7E-308 to 1.7E+308
- **Size:** 8 bytes
- **Format Specifier:** %lf

Syntax for declaring the double variables:

```
double var_name;
```

Ex:

```
double a;
```

```
double x,y;
```

Void Data Type: The void data type in C is used to specify that no value is present. It does not provide a result value to its caller. It has no values and no operations. It is used to represent nothing.

Void is used in multiple ways as function return type, function arguments as void, and [pointers to void](#).

Variables:

A **variable in C language** is the name associated with some memory location to store data of different types. There are many types of variables in C depending on the scope, storage class, lifetime, type of data they store, etc. A variable is the basic building block of a C program that can be used in expressions as a substitute in place of the value it stores. Following is the syntax for declaring the variables in C language.

`data_type variable_name; // defining single variable`

or

`data_type variable_name1, variable_name2; // defining multiple variable`

Here,

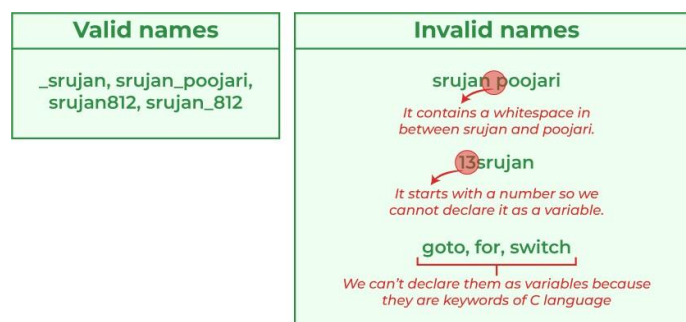
- **data_type:** Type of data that a variable can store and it is a valid data type.
- **variable_name:** Name of the variable given by the user.

Example:

```
int var; // integer variable
char a; // character variable
float fff; // float variables
```

Rules for Naming Variables in C:

1. A variable name must only contain alphabets, digits, and underscore.
2. A variable name must start with an alphabet or an underscore only. It cannot start with a digit.
3. No whitespace is allowed within the variable name.
4. A variable name must not be any reserved word or keyword.



Variable Initialization

- ✓ The process of assigning a value to a variable is called initialization.
- ✓ The process of giving a value to a variable is called initialization.
- ✓ The process of presenting a value to a variable is called initialization.
- ✓ The process of specifying a value to a variable is called initialization.
- ✓ For example:

x = 10;

in the above statement the variable is “x” and the value is “10”. Here the value 10 is given to the variable x by using assignment operator “=”. Therefore if we want to assign a value to a variable then we need an assignment operator. The variable here used “x” is called initialized variable.

For example:

int x=10;

float y=2.5;

char = 'a';

in the above statements the variables are initialized to values that are of different data type. The variables are called initialized variables or variables are said to be initialized.

Types of Variables in C

There are many types of variables in c:

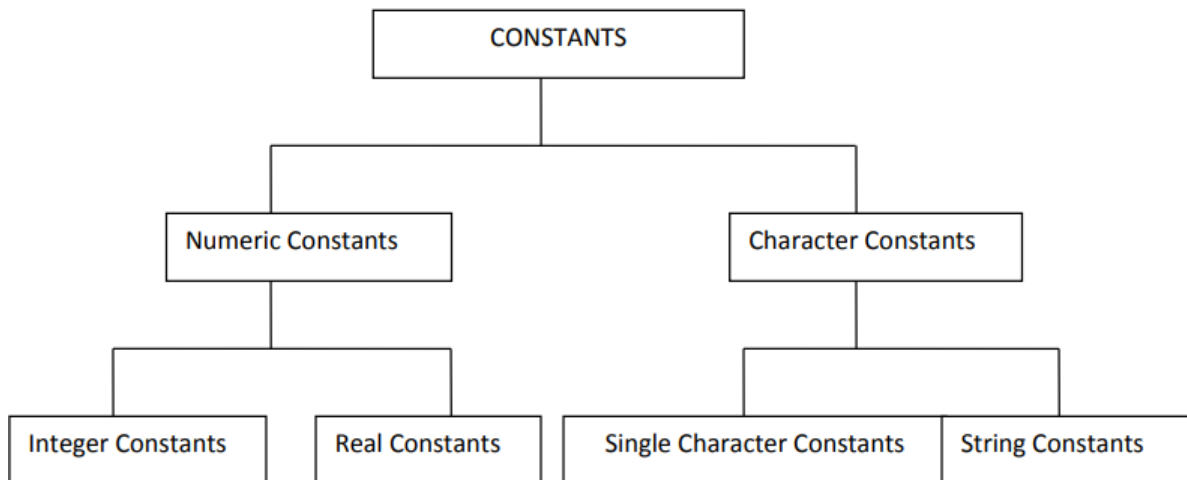
1. local variable
2. global variable
3. static variable

Constants

Constants refer to fixed values that do not change during the execution of a program.

Note: constants are also called literals.

C supports several kinds of constants.



TYPES OF C CONSTANT:

1. Integer constants
2. Real or floating point constants
3. Character constants
4. String constants
5. Backslash character constants

Integer constants:

An integer constant is a numeric constant (associated with number) without any fractional or Exponential part. There are three types of integer constants in C programming:

- decimal constant(base 10)
- octal constant(base 8)
- hexadecimal constant(base 16)

For example:

- Decimal constants: 0, -9, 22 etc
- Octal constants: 021, 077, 033 etc

- Hexadecimal constants: 0x7f, 0x2a, 0x521 etc
- In C programming, octal constant starts with a 0 and hexadecimal constant starts with a 0x

1: Decimal Integer: the rules for represent decimal integer.

- Decimal Integer value which consist of digits from 0-9.
- Decimal Integer value with base 10.
- Decimal Integer should not prefix with 0.
- It allows only sign (+,-).
- No special character allowed in this integer.

Ex: valid invalid

7 \$77

77 077

+77 7,777

2 : Octal : An integer constants with base 8 is called octal. These rules are:

- it consist of digits from 0 to 7.
- It should prefix with 0.
- It allows sign (+,-).
- No special character is allowed.

EX: VALID INVALID

0123 123 -> it because no prefix with 0

+0123 0128 -> because digits from 0 to 7.

-012

3 : Hexadecimal : An integer constant with base value 16 is called Hexadecimal.

a) It consist of digits from 0-9,a-f(capital letters & small leters.

Ex : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

b) it should prefix with 0X or 0x.

c) it allows sign (+,-).

d) No special character is allowed.

EX: 0X1a, 0x2

Floating point/Real constants:

A floating point constant is a numeric constant that has either a fractional form or an exponent

Form. For example:

-2.0

0.0000234

-0.22E-5

Character Constant:

Single Character Constant: A character constant is either a single alphabet, a single digit, a

Single special symbol enclosed within single inverted commas.

a) It is value represent in „ „, (single quote).

b) The maximum length of a character constant can be 1 character.

EX : VALID INVALID

„a“ “12”

String constant : A string constant is a sequence of characters enclosed in double quote, the characters may be letters, numbers, special characters and blank space etc

EX : “rama” , “a” , “+123” , “1-/a”

"good" //string constant

"" //null string constant

" " //string constant of six white space

"X" //string constant having single character.

"Earth is round\n" //prints string with newline

Escape characters or backslash characters:

- a) \n newline
- b) \r carriage return
- c) \t tab
- d) \v vertical tab
- e) \b backspace
- f) \f form feed (page feed)
- g) \a alert (beep)
- h) \' single quote(,,)
- i) \" double quote(“)
- j) \? Question mark (?)
- k) \\ backslash (\)

Two ways to define constant in C

There are two ways to define constant in C programming.

1. const keyword
2. #define preprocessor

1) C const keyword

The const keyword is used to define constant in C programming.

```
const float PI=3.14;
```

Now, the value of PI variable can't be changed.

```
1. #include <stdio.h>
2. #include <conio.h>
3. void main(){
4. const float PI=3.14;
5. clrscr();
6. printf("The value of PI is: %f",PI);
7. getch();
8.}
```

Output:

The value of PI is: 3.140000

2) C #define preprocessor

The #define preprocessor is also used to define constant.

C#define

The #define preprocessor directive is used to define constant or micro substitution. It can use any Basic data type.

Syntax:

```
#define token value
```

Let's see an example of #define to define a constant.

```
#include <stdio.h>
```

```
1. #define PI 3.14
2. main ()
```



```
{
3. printf("%f",PI);
4.}
```

Output:

3.140000

Input / Output (I/O) Functions : In „C“ language, two types of Input/Output functions are available, and all input and output operations are carried out through function calls. Several functions are available for input / output operations in „C“. These functions are collectively known as the standard i/o library.

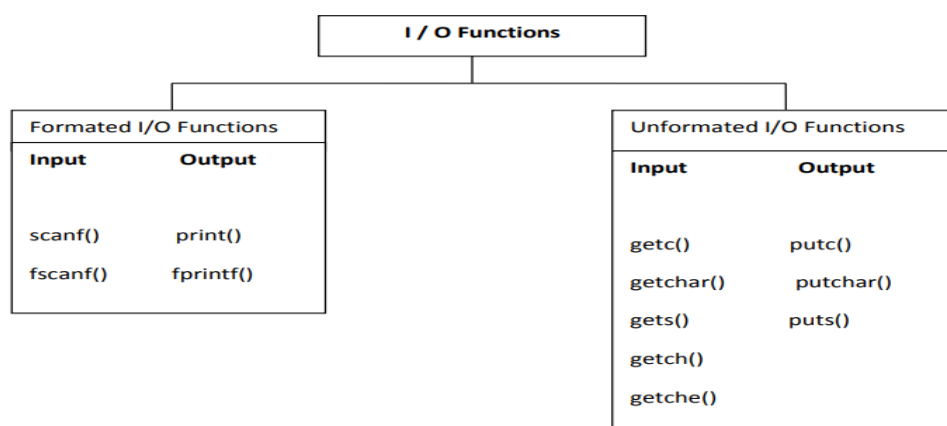
Input: In any programming language input means to feed some data into program. This can be given in the form of file or from command line.

Output: In any programming language output means to display some data on screen, printer or in any file.

The Standard Files

C programming treats all the devices as files. So devices such as the display are addressed in the same way as files and the following three files are automatically opened when a program executes to provide access to the keyboard and screen.

Input / Output functions are classified into two types



Formatted I/O Functions: formatted I/O functions operates on various types of data.

1 : printf() : output data or result of an operation can be displayed from the computer to a standard output device using the library function printf(). This function is used to print any combination of data.

Syntax : printf("control string ", variable1, variable2, -----, variablen);

Ex : printf("%d",3977); // Output: 3977

printf() statement another syntax :

Syntax : printf("fomating string");

Formatting string: it prints all the character given in double quotes (" ") except formatting Specifier.

Ex : printf(" hello ");-> hello

printf("a"); -> a

printf("%d", a); -> a value

printf("%d"); -> no display

2. scanf() : input data can be entered into the computer using the standard input „C“ library function called scanf(). This function is used to enter any combination of input.

Syntax : scanf("control string ",&var1, &var2,----, &varn);

The scanf() function is used to read information from the standard input device (keyboard).

Ex : scanf(" %d ",&a);-> hello

Each variable name (argument) must be preceded by an ampersand (&). The (&) symbol gives the meaning "address of "the variable.

Unformatted I/O functions:

a) Character I/O

b) String I/O

a) character I/O:

1. `getchar()`: Used to read a character from the standard input
2. `putchar()`: Used to display a character to standard output
3. `getch()` and `getche()`: these are used to take the any alpha numeric characters from the standard input

`getche()` read and display the character

`getch()` only read the single character but not display

4. `putch()`: Used to display any alpha numeric characters to standard output

b) String I/O:

1. `gets()`: Used for accepting any string from the standard input(stdin)

eg: `gets()`

2. `puts()`: Used to display a string or character array Eg: `puts()`

3. `Cgets()`: read a string from the console eg; `cgets(char *st)`

4. `Cputs()`: display the string to the console eg; `cputs(char *st)`

Operators : An operator is a Symbol that performs an operation. An operators acts some variables are called operands to get the desired result.

Ex : `a+b;`

Where a,b are operands and + is the operator.

Types of Operator:

- 1) Arithmetic Operators.
- 2) Relational Operators.
- 3) Logical Operators.
- 4) Assignment Operators.

- 5). Unary Operators.
- 6) Conditional Operators.
- 7) Special Operators.
- 8) Bitwise Operators.

Arithmetic Operators:

An arithmetic operator performs mathematical operations such as addition, subtraction and Multiplication on numerical values (constants and variables).

C Program to demonstrate the working of arithmetic operators

```
#include <stdio.h>

void main()

{

    int a = 9,b = 4, c;

    c = a+b;

    printf("a+b = %d \n",c);

    c = a-b;

    printf("a-b = %d \n",c);

    c = a*b;

    printf("a*b = %d \n",c);

    c=a/b;

    printf("a/b = %d \n",c);

    c=a%b;

    printf("Remainder when a divided by b = %d \n",c);

}
```

Output

$a+b = 13$

$a-b = 5$

$a*b = 36$

$a/b = 2$

Remainder when a divided by b=1

Relational Operators: A relational operator checks the relationship between two operands.

If the relation is true, it returns 1; if the relation is false, it returns value 0. Operands may be variables, constants or expressions. Relational operators are used in decision making and loops.

Operator	Meaning	Example	Return value
<	is less than	$2 < 9$	1
<=	is less than or equal to	$2 <= 2$	1
>	is greater than	$2 > 9$	0
>=	is greater than or equal to	$3 >= 2$	1
==	is equal to	$2 == 3$	0
!=	is not equal to	$2 != 2$	0

// C Program to demonstrate the working of relational operators

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int a = 5, b = 5, c = 10;
```

```
printf("%d == %d = %d \n", a, b, a == b); // true
```

```
printf("%d == %d = %d \n", a, c, a == c); // false
```

```
printf("%d > %d = %d \n", a, b, a > b); //false
```

```
printf("%d > %d = %d \n", a, c, a > c); //false
```

```
printf("%d < %d = %d \n", a, b, a < b); //false
```

```
printf("%d < %d = %d \n", a, c, a < c); //true
```

```
printf("%d != %d = %d \n", a, b, a != b); //false  
return 0;  
}
```

Output

```
5 == 5 = 1  
5 == 10 = 0  
5 > 5 = 0  
5 > 10 = 0  
5 < 5 = 0  
5 < 10 = 1  
5 != 5 = 0
```

Logical Operators.

These operators are used to combine the results of two or more conditions. An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in decision making in C programming.

Operator	Meaning	Example	Return value
&&	Logical AND	(9>2)&&(17>2)	1
	Logical OR	(9>2) (17 == 7)	1
!	Logical NOT	29!=29	0

Logical AND : If any one condition false the complete condition becomes false.

Truth Table

Op1	Op2	Op1 && Op2
true	true	true
true	false	false
false	true	false
false	false	false

Logical OR : If any one condition true the complete condition becomes true.

Truth Table

Op1	Op2	Op1 Op2
true	true	true
true	false	true
false	true	true
false	false	false

Logical Not : This operator reverses the value of the expression it operates on i.e, it makes a true expression false and false expression true.

Op1	Op1 !
true	false
false	true

// C Program to demonstrate the working of logical operators

```
#include <stdio.h>
```

```
int a = 5, b = 5, c = 10, result;
```

```
result = (a = b) && (c > b);
```

```
printf("(a = b) && (c > b) equals to %d \n", result);
```

```
result = (a = b) && (c < b);
```

```
printf("(a = b) && (c < b) equals to %d \n", result);
```

```
result = (a = b) || (c < b);
```

```
printf("(a = b) || (c < b) equals to %d \n", result);
```

```

result = (a != b) || (c < b);

printf("(a != b) || (c < b) equals to %d \n", result);

return 0;

}

```

Output

(a = b) && (c > b) equals to 1

(a = b) && (c < b) equals to 0

(a = b) || (c < b) equals to 1

(a!= b) || (c < b) equals to 0

Assignment Operators: Assignment operators are used to assign a value (or) an expression (or) a value of a variable to another variable.

Syntax : variable name=expression (or) value (or) variable

Ex : x=10;

y=a+b;

z=p;

Compound assignment operator:

„C“ provides compound assignment operators to assign a value to variable in order to assign a new value to a variable after performing a specified operation.

Operator	Example	Meaning
+=	x += y	x=x+y
-=	x -= y	x=x-y
*=	x *= y	x=x*y
/=	x /= y	x=x/y
%=	x %= y	X=x%y

Conditional Operator/ Ternary operator:

Conditional operator checks the condition and executes the statement depending of the condition. A conditional operator is a ternary operator, that is, it works on 3 operands.

Conditional operator consist of two symbols.

1:

que

stio

n

mar

k

(?).

2:

colo

n (:

).

Syntax: condition? exp1: exp2;

It first evaluate the condition, if it is true (non-zero) then the “exp1” is evaluated, if the condition is false (zero) then the “exp2” is evaluated.

Example:

```
#include<stdio.h> void main()
{
int x,y,z;
clrscr();
printf("Enter the value of a and b :");
scanf("%d %d",&x,&y);
z=((x>y)?x:y);
printf("The biggest value is %d",z); getch();
}
```

Output:

Enter the value of a and b: 125 100
The biggest value is 125

Enter the value of a and b: 25 100
The biggest value is 100

Bitwise Operators:

Bitwise operators are used to manipulate the data at bit level. It operates on integers only. It may not be applied to float. In arithmetic-logic unit (which is within the CPU), mathematical operations like: addition, subtraction, multiplication and division are done in bit-level which makes processing faster and saves power. To perform bit-level operations in C programming, bitwise operators are used.

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
<<	Shift left
>>	Shift right
~	One's complement.

Bitwise AND operator &

The output of bitwise AND is 1 if the corresponding bits of two operands is 1. If either bit of an

Operand is 0, the result of corresponding bit is evaluated to 0.

Let us suppose the bitwise AND operation of two integers 12 and 25.

12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

Bit Operation of 12 and 25

00001100

& 00011001

00001000 = 8 (In decimal)

Example #1: Bitwise AND

```
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a&b);
    return 0;
}
```

Output

Output =8

Bitwise OR operator |

The output of bitwise OR is 1 if at least one corresponding bit of two operands is 1. In C Programming, bitwise OR operator is denoted by |.

12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

Bitwise OR Operation of 12 and 25

00001100

| 00011001

00011101 = 29 (In decimal)

Example #2: Bitwise OR

```
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a|b);
    return 0;
}
```

Output

Output =29

Bitwise XOR (exclusive OR) operator ^

The result of bitwise XOR operator is 1 if the corresponding bits of two operands are opposite. It

is denoted by ^.

12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

Bitwise XOR Operation of 12 and 25

00001100

| 00011001

00010101 = 21 (In decimal)

Example #3: Bitwise XOR

```
#include <stdio.h>
int main()
{
    int a = 12, b = 25;
    printf("Output = %d", a^b);
    return 0;
}
```

Output

Output = 21

Bitwise complement operator ~

Bitwise compliment operator is an unary operator (works on only one operand). It changes 1 to 0

and 0 to 1. It is denoted by ~.

35 = 00100011 (In Binary)

Bitwise complement Operation of 35

~ 00100011

11011100 = 220 (In decimal)

There are two Bitwise shift operators in C programming:

□ Right shift operator

□ Left shift operator.

Right Shift Operator

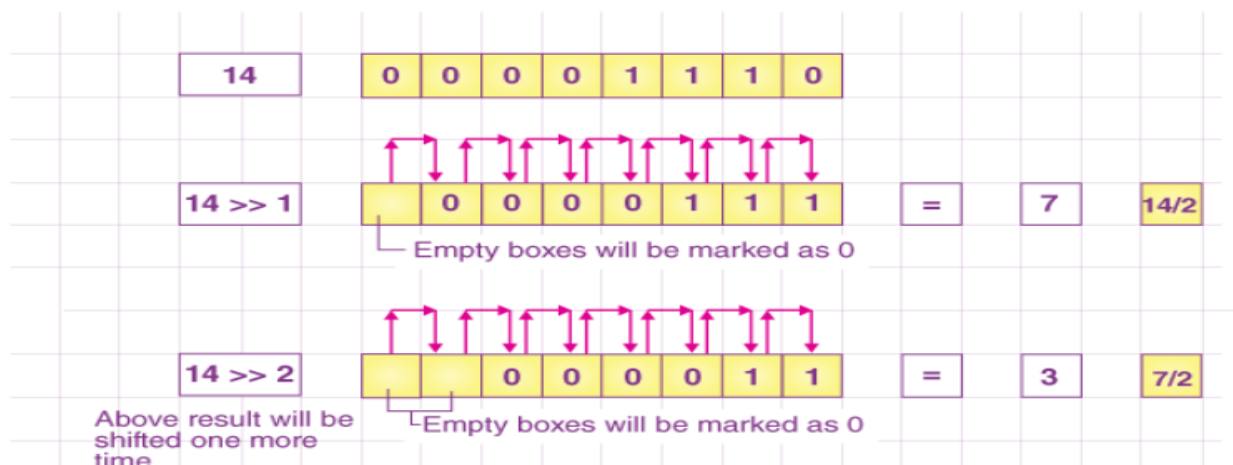
Right shift operator shifts all bits towards right by certain number of specified bits. It is denoted by $\gg$.

syntax: `variable_name>>number_of_positions`

For Example,

Let us consider the number 14. Now, the binary representation of this number is 00001110 in 8 bits (for convenience, let us consider it this way).

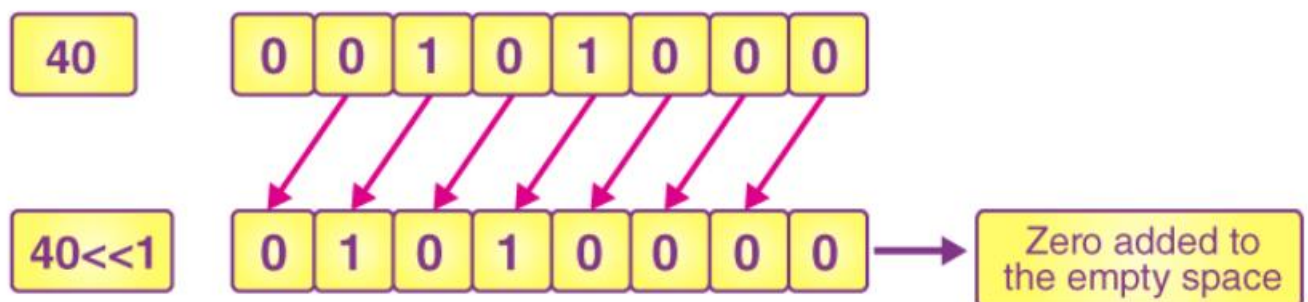
In this case, the $14 \gg 1$ will shift this binary sequence by a single position to the right. Like,



Left Shift Operator

Left shift operator shifts all bits towards left by certain number of specified bits. It is denoted by $\ll$.

Syntax: `variable name<<number_of_positions`



In this example, you can clearly notice that all the bits have shifted to the left once we performed the left shift operation on the binary value. The empty space thus created is now filled with zeroes (0).

Thus, the value obtained from $40 \ll 1$ is 01010000, and 80 is the decimal equivalent of this binary value.

Special Operators

1) **Comma Operator:** The comma operator is used to separate the statement elements such as variables, constants or expressions, and this operator is used to link the related expressions together, such expressions can be evaluated from left to right and the value of right most expressions is the value of combined expressions

Ex : `val(a=3, b=9, c=77, a+c)`

First signs the value 3 to a, then assigns 9 to b, then assigns 77 to c, and finally 80 (3+77) to value.

2) **Sizeof Operator :** The `sizeof()` is a unary operator, that returns the length in bytes of the specified variable, and it is very useful to find the bytes occupied by the specified variable in the memory.

Syntax : `sizeof(variable-name);`

`int a;`

`Ex : sizeof(a); //OUTPUT-----2bytes`

Example #6: `sizeof Operator`

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int a, e[10];
```

```
float b;
```

```
double c;
```

```
char d;
```

```
printf("Size of int=%lu bytes\n",sizeof(a));
```

```
printf("Size of float=%lu bytes\n",sizeof(b));
```

```
printf("Size of double=%lu bytes\n",sizeof(c));
```

```
printf("Size of char=%lu byte\n",sizeof(d));
```

```
printf("Size of integer type array having 10 elements = %lu bytes\n", sizeof(e));
```

```
return 0;
```

```
}
```

Output

Size of int = 4 bytes

Size of float = 4 bytes

Size of double = 8 bytes

Size of char = 1 byte

Size of integer type array having 10 elements = 40 bytes

Expressions

Expressions: An expression is a combination of operators and operands which reduces to a single value. An operator indicates an operation to be performed on data that yields a value. An operand is a data item on which an operation is performed.

A simple expression contains only one operator.

Introduction To Programming

Ex: $3+5$ is a simple expression which yields a value 8, $-a$ is also a single expression.

A complex expression contain more than one operator.

Ex: complex expression is $6+8*7$.

Ex; Algebraic Expressions

1: ax^2+bx+c

2: $a+bx$

3: $4ac/b$

4: x^2/y^2-1

C-expression

1: $a*x*x+b*x+c$

2: $a+b*x$.

3: $4*a*c/b$.

4: $x*x/y*y-1$

Operator Precedence: Arithmetic Operators are evaluated left to right using the Precedence of operator when the expression is written without the paranthesis. They are two Levels of arithmetic operators in C.

1: High Priority $*$ / $\%$

2: Low Priority $+$ $-$.

Arithmetic Expression evaluation is carried out using the two phases from left to right.

1: First phase: The highest priority operator are evaluated in the 1st phase.

2: Second Phase: The lowest priority operator are evaluated in the 2nd phase.

Ex: $a=x-y/3+z*2+p/4$.

$x=7$, $y=9$, $z=11$, $p=8$.

$a=7-9/3+11*2+8/4$.

1st phase:

1: $a=7-3+11*2+8/4$

2: $a=7-3+22+8/4$

3: $a=7-3+22+2$

2nd phase:

1: $a=4+22+2$

2: $a=26+2$

3: $a=28$

The order of evaluation can be changed by putting parenthesis in an expression.

Ex: $9-12/(3+3)*(2-1)$

Whenever parentheses are used, the expressions within parenthesis highest priority. If two or more sets of parenthesis appear one after another. The expression contained in the left-most set is evaluated first and the right-most in the last.

1st phase:

1: $9-12/6*(2-1)$

2: $9-12/6*1$

2nd phase:

1: $9-2*1$

2: $9-2$.

3rd phase:

1: 7.

Rules for Evaluation of Expression:

- 1: Evaluate the sub-expression from left to right. If parenthesized.
- 2: Evaluate the arithmetic Expression from left to right using the rules of precedence.
- 3: The highest precedence is given to the expression with in parenthesis.
- 4: When parenthesis are used, the expressions within parentheses assume highest priority.
- 5: Apply the associative rule, if more operators of the same precedence occurs.

Operator Precedence and Associativity:

The precedence and associativity of various operators in C.

Operator	Description	Precedence	Associativity
() []	Function call Square brackets.	1	L-R (left to right)
+ - ++ -- ! ~ * & sizeof	Unary plus Unary minus Increment Decrement Not operator Complement Pointer operator Address operator Sizeof operator	2	R-L (right to left)
* / %	Multiplication Division Modulo division	3	L-R (left to right)
+ -	Addition Subtraction	4	L-R (left to right)
<< >>	Left shift Right shift	5	L-R (left to right)

< <= > >=	Relational Operator	6	L-R (left to right)
= = !=	Equality Inequality	7	L-R (left to right)
&	Bitwise AND	8	L-R (left to right)
^	Bitwise XOR	9	L-R (left to right)
	Bitwise OR	10	L-R (left to right)
&&	Logical AND	11	L-R (left to right)
	Logical OR	12	L-R (left to right)
?:	Conditional	13	R-L (right to left)
= *= /= %= += -= &= ^= <<= >>=	Assignment operator	14	R-L (right to left)
,	Comma operator	15	L-R (left to right)

Type Conversion/Type casting:

Type conversion is used to convert variable from one data type to another data type, and after

type casting compiler treats the variable as of new data type.

For example, if you want to store a 'long' value into a simple integer then you can type cast 'long' to 'int'. You can convert the values from one type to another explicitly using the cast operator. Type conversions can be implicit which is performed by the compiler automatically, or it can be specified explicitly through the use of the cast operator.

Syntax:

(type_name) expression;

Without Type Casting:

1. `int f= 9/4;`
2. `printf("f : %d\n", f );//Output: 2`

With Type Casting:

1. `float f=(float) 9/4;`
2. `printf("f : %f\n", f );//Output: 2.250000`

Type Casting - C Programming

Type casting refers to changing a variable of one data type into another. The compiler will automatically change one type of data into another if it makes sense. For instance, if you assign an integer value to a floating-point variable, the compiler will convert the int to a float. Casting allows you to make this type conversion explicit, or to force it when it wouldn't normally happen.

Type conversion in c can be classified into the following two types:

1. Implicit Type Conversion

When the type conversion is performed automatically by the compiler without programmers Intervention, such type of conversion is known as implicit type conversion or type promotion.

```
int x;
for(x=97; x<=122; x++)
{
    printf("%c", x); /*Implicit casting from int to char thanks to %c*/
}
```

2. Explicit Type Conversion

The type conversion performed by the programmer by posing the data type of the expression of Specific type is known as explicit type conversion. The explicit type conversion is also known as type casting.

Type casting in c is done in the following form:

(Data type) expression;

Where, data type is any valid c data type, and expression may be constant, variable or expression.

For example,

```
int x;
for(x=97; x<=122; x++)
{
```



```
printf("%c", (char)x); /*Explicit casting from int to char*/  
}
```

The following rules have to be followed while converting the expression from one type to another to avoid the loss of information:

All integer types to be converted to float.

All float types to be converted to double.

All character types to be converted to integer.

Example

Consider the following code:

```
int x=7, y=5 ;
```

```
float z;
```

```
z=x/y; /*Here the value of z is 1*/
```

If we want to get the exact value of 7/5 then we need explicit casting from int to float:

```
int x=7, y=5;
```

```
float z;
```

```
z = (float)x/(float)y; /*Here the value of z is 1.4*/
```